

Push Authenticator — Mobile App Integration Guide

Push Authenticator — Mobile App Integration Guide

****Project:**** push-authenticator-spi (Keycloak custom SPI)

****Delivery mechanism:**** Direct WebSocket (no Firebase / FCM involved)

****Test environment:**** Local Wi-Fi network

1. Test Environment Details

| Item | Value |

|---|---|

| Server IP (same Wi-Fi) | `192.168.10.106` *(confirm this — see note below)* |

| Keycloak port | `8280` |

| WebSocket port | `8090` |

| Realm | `test3` |

> **△ Please confirm the IP before starting.** Run `ipconfig` (Windows) or `ifconfig` / `ip addr` (Mac/Linux) on the server machine and look for the Wi-Fi/LAN adapter IP (usually `192.168.x.x` or `10.x.x.x`). Use that exact value everywhere `` appears below.

****Before writing any app code****, verify connectivity from your phone's browser (same Wi-Fi):

...

http://<server-ip>:8280/realms/test3

...

If this loads a JSON response with realm info, networking is confirmed working.

****Important:**** Always use ``, never `localhost` or `127.0.0.1` — those would point to the phone itself, not the server.

2. High-Level Flow

...

1. Scan QR code
2. Call Device Registration API → receive { deviceSecret, wsUrl }
3. Store deviceSecret securely (Keystore / Keychain) — shown only once
4. Open WebSocket immediately using wsUrl + deviceSecret
5. Keep the WebSocket open permanently; reconnect on any drop
6. Listen for LOGIN_APPROVAL messages on the socket
7. On receiving one: show local biometric/PIN prompt → call approve or reject URL

...

There is ****no Firebase, no push notification service, and no polling**** on the mobile side — the WebSocket is the entire delivery channel. If it's not connected, the app will not

receive login prompts.

3. API 1 — Device Registration

Called immediately after scanning the QR code.

****Request****

PUT http://<server-ip>:8280/realms/test3/push/device/register/{id}

Content-Type: application/json

`{id}` and `authToken` below come from the scanned QR JSON — never hardcode them.

****Example QR JSON (what scanning produces):****

```json

```
{
 "type": "PUSH_DEVICE_REGISTRATION",
 "request_url": "http://<server-ip>:8280/realms/test3/push/device/register/{id}",
 "request_type": "PUT",
 "auth_token": "EOXWEOQS7OWCRWNCHHOFKJGMV7WNGJSL",
 "expires_in": 600,
 "user": {
 "username": "suraj.biswal1",
 "realm": "test3",
 "given_name": "Suraj",
 "family_name": "Biswal"
 },
 "request_device_information": {
 "deviceId": "",
 "platform": "",
 "capabilities": { "pin": false, "face": true, "fingerprint": true }
 }
}
```

```

****Request body parameters:****

Param	Type	Required	Description
-------	------	----------	-------------

---	---	---	---
-----	-----	-----	-----

`authToken`	string	yes	Copy exactly from QR's `auth_token`. One-time use. Expires in `expires_in` seconds (600 = 10 min).
-------------	--------	-----	--

`deviceId`	string	yes	App-generated stable unique ID (e.g. `UUID.randomUUID()`) stored locally). **Must not change** across app restarts.
------------	--------	-----	--

`deviceName`	string	yes	Human-readable label, e.g. `"Suraj's Galaxy S23"`.
`platform`	string	yes	`"android"` or `"ios"`.
`publicKey`	string	yes	Send `""` for now — reserved for future use.
`capabilities`	object	yes	`{ "pin": bool, "face": bool, "fingerprint": bool }` based on what the device actually supports/has enrolled.

****Example request body:****

```
```json
{
 "authToken": "EOXWEOQS7OWCRWNCHHOFKJGMV7WNGJSL",
 "deviceId": "a1b2c3d4-uuid-generated-by-app",
 "deviceName": "Suraj's Galaxy S23",
 "platform": "android",
 "publicKey": "",
 "capabilities": { "pin": false, "face": true, "fingerprint": true }
}
```
```

****Success response — `200 OK`:****

```
```json
{
 "status": "SUCCESS",
 "message": "Device Registered Successfully",
 "deviceSecret": "gT4yeIYYCeyTbd6dUod6WmSWgkCVnI870jV8SQPCXKk",
 "wsUrl": "ws://<server-ip>:8090/?deviceId=a1b2c3d4-uuid-generated-by-app"
}
```
```

> ****⚠ CRITICAL:**** `deviceSecret` is shown ****exactly once****, in this response only. Store it immediately in:

> - ****Android:**** Keystore-backed encrypted storage (e.g. `EncryptedSharedPreferences`)

> - ****iOS:**** Keychain

>

> Never store it in plain SharedPreferences / UserDefaults / logs. If lost, the device must be re-registered from a fresh QR scan (API 6 can help if `deviceId` is still known).

****Failure response — `400 Bad Request`:****

```
```json
{ "status": "FAILED", "message": "Invalid, expired, or already used registration link" }
```
```

Causes: QR expired (>10 minutes old), already used once, or reused an old QR.
Solution: scan a fresh QR.

4. API 2 — WebSocket Connection (the always-on channel)

****Connect using the `wsUrl` from API 1, appending the secret:****

...

```
{wsUrl}&secret={deviceSecret}
```

...

****Full example:****

...

```
ws://192.168.10.106:8090/?deviceId=a1b2c3d4-uuid-generated-by-app&secret=gT4yeIYYCeyTbd6dUod6WmSWgkCVnl870jV8SQPCXKk
```

...

When to connect (implement all of these):

1. ****Immediately**** after successful registration (API 1 success).
2. ****On every app launch / cold start**** — using the stored `deviceId` + `deviceSecret`. No re-registration needed.
3. ****On network change**** — Wi-Fi ↔ cellular switch, airplane mode toggle.
4. ****On any unexpected disconnect**** — reconnect with exponential backoff (e.g. 2s → 4s → 8s → 16s, capped around 60s).
5. Ideally run inside a ****foreground service**** (Android) or appropriate ****background execution mode**** (iOS) so the connection survives backgrounding. Neither OS guarantees indefinite background socket survival — this is a known tradeoff of a pure WebSocket design (no OS-level push wake, unlike Firebase).

Connection failure codes:

| Code | Meaning | App should do |
|------|---------|---------------|
|------|---------|---------------|

| | | |
|-----|-----|-----|
| --- | --- | --- |
|-----|-----|-----|

| | | |
|--------|---------------------------------------|---------------------------|
| `4000` | Missing `deviceId` or `secret` in URL | Fix the connection string |
|--------|---------------------------------------|---------------------------|

| | | |
|--------|---|--|
| `4001` | Invalid `deviceId`/`secret` combination | Treat device as unregistered — prompt user to re-register via QR |
|--------|---|--|

****No acknowledgment message is sent on successful connect**** — the WebSocket simply opens. Nothing else to parse at that point.

Message received on this socket (push-style, event-driven, no polling):

```
```json
```

```
{
```

```
"notification": {
```

```
"title": "Login Approval Request",
```

```
"body": "Approve login for suraj.biswal1?"
```

```
},
```

```
"data": {
```

```
"type": "LOGIN_APPROVAL",
```

```
"sub_type": "FACE",
```

```
"transactionId": "65732548-e0cc-4f8d-8e4d-e771b17a43bd",
```

```
"approve_url": "http://192.168.10.106:8280/realms/test3/push/login/approve/{transactionId}/{authToken}",
"reject_url": "http://192.168.10.106:8280/realms/test3/push/login/reject/{transactionId}/{authToken}",
"request_type": "POST"
}
}
...
```

| Field | Meaning |

|---|---|

| `notification.title` / `notification.body` | Display text for the prompt shown to the user. |

| `data.sub\_type` | Which local auth method to trigger: `FACE`, `FINGERPRINT`, `PIN`, or `NONE` — decided server-side from the `capabilities` sent at registration. |

| `data.transactionId` | Unique identifier for this specific login attempt. |

| `data.approve\_url` / `data.reject\_url` | **Fully-formed URLs** — already contain the one-time token. Call exactly as received; do not construct or modify. |

| `data.request\_type` | Always `POST` — the HTTP method to use for approve/reject. |

---

### ## 5. API 3 — Approve Login

Called after the user confirms via local biometric/PIN prompt.

...

POST {approve\_url exactly as received in the WebSocket message}

...

No request body. No auth header required (token is embedded in the URL).

**Success:**

```
```json
```

```
{ "status": "APPROVED" }
```

```
```
```

**Failure — `400`:**

```
```json
```

```
{ "status": "FAILED", "message": "Invalid or expired request" }
```

```
```
```

---

### ## 6. API 4 — Reject Login

...

POST {reject\_url exactly as received in the WebSocket message}

...

**Success:**

```
```json
{ "status": "REJECTED" }
```
```

---

## 7. API 5 — List Registered Devices (optional — e.g. "manage devices" screen)

```

GET http://<server-ip>:8280/realms/test3/push/device/list

Authorization: Bearer {user's access token}

```

**\*\*Response:\*\***

```
```json
[
{
"deviceId": "a1b2c3d4-uuid-generated-by-app",
"deviceName": "Suraj's Galaxy S23",
"platform": "android",
"registered": true,
"capabilities": "{\"pin\":false,\"face\":true,\"fingerprint\":true}",
"createdAt": "2026-07-17 11:22:40.558704"
}
]
```
```

---

## 8. API 6 — Unregister a Device

```

DELETE http://<server-ip>:8280/realms/test3/push/device/unregister/{deviceId}

Authorization: Bearer {user's access token}

```

**\*\*Success:\*\***

```
```json
{ "status": "SUCCESS", "message": "Device removed successfully" }
```
```

> **\*\*App must also\*\***, on this action: delete the locally stored `deviceSecret`, and close the WebSocket connection. The secret is now invalid server-side — any reconnect attempt will get closed with code `4001`.

---

## 9. API 7 — Reissue Secret (app reinstalled / data cleared, `deviceId` still known)

```

POST http://<server-ip>:8280/realms/test3/push/device/update

Content-Type: application/json

```

**\*\*Body:\*\***

```json

```
{
  "deviceId": "a1b2c3d4-uuid-generated-by-app",
  "platform": "android",
  "deviceName": "Suraj's Galaxy S23"
}
```

```

**\*\*Success — returns a brand-new secret; the old one is now invalid:\*\***

```json

```
{
  "status": "SUCCESS",
  "message": "Device updated successfully",
  "deviceSecret": "<new secret>",
  "wsUrl": "ws://<server-ip>:8090/?deviceId=a1b2c3d4-uuid-generated-by-app"
}
```

```

If `deviceId` is unknown/lost too, a full fresh QR registration (API 1) is required instead.

---

**## 10. Getting a Test Access Token (for APIs requiring `Authorization: Bearer`)**

```

POST http://<server-ip>:8280/realms/test3/protocol/openid-connect/token

Content-Type: application/x-www-form-urlencoded

client_id=account-console

grant_type=password

username=suraj.biswal1

password=<their actual password>

```

Copy `access\_token` from the response, use as `Authorization: Bearer <access\_token>` in APIs 5 and 6.

---

**## 11. Rules to Follow (Non-negotiable)**

- **\*\*The WebSocket must stay open at all times\*\*** the app expects to receive login prompts. There is no fallback wake mechanism. If disconnected when a login happens, the prompt is simply missed and the browser login attempt will time out on its own.

- **\*\*Reconnect logic is mandatory\*\***, not optional — build it into the core networking layer from day one, with backoff.

- **`deviceId` must remain stable** across app restarts — do not regenerate it on every launch; it's the server's lookup key for routing messages to this exact device.
- **`deviceSecret` is a one-time reveal** — treat it like a password: encrypted storage only, never logged, never transmitted except as the WebSocket connection's `secret` query parameter.
- **Gate approve/reject behind the local biometric/PIN prompt** matching `sub\_type` — never call `approve\_url` without the user actually confirming on-device.
- **Always use the shared server IP**, never `localhost` — the phone and server are separate devices on the same network.

---

## ## 12. Quick Reference — All Endpoints

#	Purpose	Method	Path
1	Register device	PUT	`/realms/test3/push/device/register/{id}`
2	WebSocket connect	WS	`ws://<ip>:8090/?deviceId={id}&secret={secret}`
3	Approve login	POST	`/realms/test3/push/login/approve/{transactionId}/{authToken}`
4	Reject login	POST	`/realms/test3/push/login/reject/{transactionId}/{authToken}`
5	List devices	GET	`/realms/test3/push/device/list`
6	Unregister device	DELETE	`/realms/test3/push/device/unregister/{deviceId}`
7	Reissue secret	POST	`/realms/test3/push/device/update`
—	Get access token	POST	`/realms/test3/protocol/openid-connect/token`

All paths are relative to `http://<server-ip>:8280` unless marked WS.